EC8206 — Functional Programming Project

Adeesha M.G.P | EG/2021/4385

1. Design, execution, and sample results

This project interprets arithmetic expression trees with variables, local bindings, and conditional expressions. `Expr.hs` contains the pure implementation; `Main.hs` demonstrates it; `Tests.hs` checks it; `comparison.py` is a runnable imperative counterpart. No external runtime packages are needed: GHC's `base` and Python's standard library suffice. From the submission folder, run `make test` and `make demo`. Without Make, run `runghc -iProject Project/Tests.hs` and `runghc -iProject Project/Main.hs`.

The syntax separates arithmetic expressions from Boolean conditions:

```
data Expr = Lit Double | Var String
  | Add Expr Expr | Sub Expr Expr | Mul Expr Expr | Div Expr Expr
  | Let String Expr Expr | If BExpr Expr Expr
data BExpr = BLit Bool | Less Expr Expr | Equal Expr Expr
  | And BExpr BExpr | Or BExpr BExpr | Not BExpr
type Env = [(String, Double)]
eval :: Env -> Expr -> Either String Double
```

The environment's first matching binding wins. A `Let` definition is evaluated in the outer environment; its body receives a new extended environment. Binary operands are evaluated left to right and the first error propagates. `If` selects one branch; `And` and `Or` short-circuit. Literals, environment values, and arithmetic results must be finite. NaN, infinity, and overflow return `Left`; ordinary floating-point rounding and underflow remain possible.

The following expected and actual results are checked by `Main.hs`, with `x = 5, y = 2`. Values below abbreviate `Right`; errors abbreviate `Left`.

Expression	Expected	Actual
2 + 3 * 4	14	14
(x - 1) / y	2	2
10 / (x - 5)	Division by zero	Division by zero
missing + 1	Undefined variable: missing	Undefined variable: missing
let a = 3 in a*a + x	14	14
let x = 10 in x + y	12	12
(let x = 10 in x) + x	15	15
if true then 7 else 1/0	7	7
if false && 1/0 < 2 then 1 else 9	9	9
let unused = 1/0 in 7	Division by zero	Division by zero

Rows 6–7 distinguish shadowing from mutation. Rows 8–9 exercise skipped computation, while row 10 confirms that a binding definition is still evaluated even when the body does not use it. The complete captured run is in `Evidence/Demo.txt`.

2. Higher-order processing and honest simplification

`evalAll env = map (eval env)` demonstrates partial application: supplying only the environment produces a function of type `Expr -> Either String Double`. `evalBatch` folds over those outcomes, retaining successful values in order. `successfulExpressions` evaluates each input once in its pipeline, then filters successful pairs and maps them back to expressions. `summarise` partitions outcomes into ordered values and messages; errors are retained rather than silently disappearing from the report.

For the ten sample expressions, the report contains **7 successes and 3 failures**. Successful values are `[14, 2, 14, 12, 15, 7, 9]`; failures are division by zero, the undefined variable, and division by zero, in that order. Empty batches have zero counts and empty lists. These behaviors have assertions in the test suite.

Rewriting uses the higher-order traversal `rewrite :: (Expr -> Expr) -> (BExpr -> BExpr) -> Expr -> Expr`. The traversal rebuilds children first, then applies a rule to their parent. Every syntax constructor has an explicit traversal case. Rule functions have a fallback because leaving a node unchanged is an intentional result.

Two different simplification contracts are exposed:

Function	Contract
simplify	Algebraic demonstration: x+0, x*1, x*0, and additional identities. Discarded terms are assumed to succeed.
simplifySafe	Conservative optimization: preserves evaluated terms unless successful evaluation independent of external bindings justifies discarding them.

For example, with no binding for `missing`:

```
eval [] (Mul (Var "missing") (Lit 0))       = Left ...
eval [] (simplify (Mul (Var "missing") (Lit 0))) = Right 0
eval [] (simplifySafe (Mul (Var "missing") (Lit 0))) = Left ...
```

Multiplication's annihilator is a valid numerical identity, but strict evaluation can fail while computing the discarded operand. Thus the first simplifier meets the algebraic demonstration contract without claiming general error preservation. The safe pass does not rewrite an unknown variable multiplied by zero. It does fold `(2+3)*4` to 20, eliminate identities such as `x/1`, and simplify closed local bindings when their evaluation succeeds.

Boolean rewrite order matters too: `false && error` may become false, whereas `error && false` must still fail. The tests include both directions. Constant branch elimination is safe when the original evaluator already skips that branch.

The safe strategy is intentionally conservative. Re-evaluating closed subtrees can repeat work; a production optimizer would use a more efficient analysis. No text parser is implemented: examples construct typed trees directly. Floating point equivalence is numerical equality, including treating signed zeros as equal; there is no claim that printed representations or IEEE sign bits are preserved.

3. Comparison with the imperative implementation

The included Python implementation uses tuple tags, explicit branching, exceptions, dictionary environments, and a loop that appends batch results. It has the same finite-number policy and evaluation order as Haskell. Its local binding deliberately implements correct scoping rather than illustrating a bug:

```
if tag == "let":
    value = evaluate(env, node[2])
    local_env = env.copy()
    local_env[node[1]] = value
    return evaluate(local_env, node[3])
```

The corresponding Haskell case constructs an extended list without changing the original environment:

```
eval env (Let name definition body) = do
  value <- eval env definition
  eval ((name, value):env) body
```

Aspect	Haskell implementation	Python implementation
Failure channel	Either String Double in the return type	EvaluationError raised internally, caught at the batch boundary
Propagation	Either bind/applicative stops at the first Left	An exception unwinds recursive calls
Local scope	Prepend a binding to an immutable list	Copy a dict, then assign a local binding
Batch processing	map, filter, foldr, and partitionEithers	for loop and append to a mutable list
Syntax checking	Constructors constrain child types and arity	Tuple tags and fields are checked during execution

Haskell's return type prevents directly adding a numeric result to an unhandled `Either`. It does not force exhaustive handling in every conceivable caller: incomplete pattern matches remain possible, which is why warnings are promoted to errors. Similarly, Python exceptions are a legitimate propagation mechanism; the difference is their visibility in this implementation's interface, not that exceptions cannot be handled correctly.

`Add (Lit 1)` is a partially applied constructor function, not an `Expr` with a missing field. It is rejected when supplied where an expression is required. Separating `BExpr` from `Expr` also rejects `If (Lit 1) ...` as a numeric condition. These structural guarantees do not establish that the arithmetic or scoping implementation is correct. A wrong subtraction in the `Add` case can still type check; behavioral tests are necessary.

The Python version demonstrates that an imperative implementation can also preserve scope. Copying a dictionary has a cost proportional to the current environment size. Prepending a Haskell binding is constant time and safely shares the immutable tail, although lookup in that association list is linear. The comparison makes no benchmark claim and does not generalize these choices to all Python or Haskell programs.

`python3 -B Project/comparison.py --parity` runs eight unittest methods, including 16 sample subcases, then compares 16 actual Haskell and Python outcomes. All outcomes agree, including shadowing, errors, skipped branches, overflow, NaN, and infinity. An additional Python assertion checks that the input environment remains unchanged.

4. Reflection, validation, and limitations

Purity. Evaluation, rewriting, and batch processing depend on explicit arguments and perform no I/O. Errors are return values; printing and process exits belong to entry points. Purity supports repeated evaluation and before/after comparisons, but does not imply that every rewrite preserves meaning.

Immutability. An extended environment shares the original immutable list as its tail; rewriting constructs nodes or shares existing subtrees. Both trees remain available for comparison. `(let x=10 in x)+x = 15`, with outer `x=5`, checks that a binding does not leak into its sibling expression.

Type-driven modeling. Constructors encode syntax shapes and separate arithmetic from Boolean expressions. Zero divisors, missing variables, and overflow still need runtime checks. Pattern coverage prevents omitted syntax cases, but does not prove termination, resource safety, or correct semantics.

Executed validation

All Haskell entry points compiled under GHC 9.6.6 with `-Wall -Wincomplete-patterns -Werror`. The lab files have **27 checks each**; the interpreter test runner has **45 checks**; the demo validates **10 examples**. Every check passed. The generated corpus contains 500 bounded expression trees: preservation is checked in three environments (1,500 comparisons), and idempotence and non-growth are checked on all 500 trees. Of the 1,500 evaluated cases, **912 return errors**, so preservation testing is not limited to successes. These are finite checks, not a proof for every possible input.

Four deliberate mutations were applied only to temporary copies: applying a function twice rather than thrice, using an incorrect grade threshold, subtracting in the addition evaluator, and substituting algebraic rewriting for the safe pass. All four caused failing checks and exit status 1. Logs are included in `Evidence/`. The Python suite and the 16-case cross-language comparison also passed.

Scope and remaining review

The labs retain the common scaffold's signatures and sentinel conventions. `average [] = 0`, `myMaximum [] = minBound`, and `treeMax Leaf = minBound` are documented conventions; optional `Maybe` helpers express absence separately. Lab sums use bounded `Int` and do not detect overflow. Interpreter operations use finite `Double` values, with normal rounding and underflow. Testing covers bounded trees, not cyclic or infinite structures or exhausted machine resources.

The original project brief was not supplied. Scope was inferred from the lecture decks and reference submissions; confirm the rubric, required signatures, report limit, and permitted assistance. `Let` and `If` demonstrate scope and short-circuit behavior.

References and assistance

- EC8206 lecture decks: Higher-Order Functions and Pattern Matching, University of Ruhuna, supplied with this task.
- G. Hutton, Programming in Haskell, 2nd edition, Cambridge University Press, 2016: defining functions, higher-order functions, and expression syntax.
- GHC User's Guide, warning options: https://downloads.haskell.org/ghc/9.6.6/docs/users_guide/using-warnings.html
- Python documentation, unittest: https://docs.python.org/3/library/unittest.html

AI assistance: OpenAI Codex assisted in generating the source, tests, report, and supporting materials, and executed the documented cloud validation. Friends' submissions were inspected as reference material and are not included. The student should review and understand the work and follow the lecturer's assistance policy.